`diffThreshold` Sensitivity Sweep of the Post-Fix Hybrid Mandelbrot Renderer

Matias Saavedra

Tuesday 26th May, 2026

Resumen

This report sweeps the `diffThreshold` parameter of `mandelHybrid` across the values $\{0.05, 0.1, 0.2, 0.3, 0.5\}$ on the post-fix binary, measuring how the four-corner uniformity heuristic's only tunable knob affects end-to-end wall time, total work generated, and the worst-case region. Three configurations are exercised — `CPU12` (CPU-only baseline), `GPU` (single GPU thread), and `GPU+11CPU` (best hybrid) — with three reps at each `diffT`. The headline result is that the hybrid sweet spot sits at `diffT` = 0.1 at 52.80 s mean, a 3.3% improvement over the 0.5 baseline (54.61 s), with a non-monotonic tail: dropping further to `diffT` = 0.05 does *worse* (53.45 s with much higher variance, σ=1.26 s vs. 0.50 s at 0.1). Pure `GPU` improves monotonically (-0.5% across the full range) and `CPU12` is essentially invariant. The most surprising finding, revealed by a `quiet=0` rerun at each `diffT`, is that **the worst-case CPU region barely moves at all**: it remains a single 960×540 region taking 14.0–15.5 s regardless of `diffT`. The heuristic cannot subdivide this region at any threshold because all four corners lie inside the Mandelbrot set (`iter`=`MAXITER`), making the corner-spread identically zero. The improvement from tightening `diffT` is therefore not coming from shrinking the binding outlier — it is coming from better load balance among the other 5,300+ leaves.

1. Setup

The four-corner subdivision rule used by `MandelRegion::examine()` is:

Código 1: mandelregion.cpp – the decision rule under study.

```
// either compute the pixels or break the region in 4 pieces
if (maxIter - minIter < diffThresh * maxIter
   || pixelsX * pixelsY < pixelSizeThresh)
 {
  compute (onGPU);
  ownerFrame->regionComplete ();
 }
else
 {
  // ... four-way split ...
 }
```

At `diffThresh` $\to$ 1 the rule almost always passes (everything looks uniform; only the pixel-size floor causes splits). At `diffThresh` $\to$ 0 the rule almost never passes (only

identical-corner regions skip splitting; the pixel floor saturates the behaviour). The interesting regime is in between.

1.1. Configurations

- **Hardware and workload**: identical to the previous two reports — i7-9750H + GTX 1660 Ti Max-Q, 100 frames at 1920×1080 from the standard `spec.in`, post-fix binary on branch `examine_minmax_bugfix`, default `pixelSizeThresh` = 32,768.
- **Threshold sweep**: $\{0.05, 0.1, 0.2, 0.3, 0.5\}$. The value 0.5 is the legacy default; it was re-used from the `sweep_results_postfix` run rather than re-measured, to save 9 redundant runs.
- **Configurations**: three labels per `diffT` — `CPU12` (`numThreads` = 12, `gpuEnable` = 0), `GPU` (`numThreads` = 1, `gpuEnable` = 1), and `GPU+11CPU` (`numThreads` = 12, `gpuEnable` = 1). The pure-GPU and best-hybrid endpoints bracket the workload, and the CPU-only baseline reports whether the effect is GPU-specific or general.
- **Reps and timing**: three reps per `(config, diffT)` pair, measured with the same end-to-end `clock_gettime` bracket as the Fig 11.15 sweep (`[total_elapsed_s]` line in the stderr log).

A second, smaller pass added one `quiet=0` run per `diffT` of `GPU+11CPU` so that the per-region `[CPU region NNNN] size WxH compute Z.ZZZ ms` lines could be parsed for max-region statistics. This added 5 runs.

2. Results

2.1. Wall Time vs. diffT

Tabla 1: Mean wall-clock time per configuration and `diffT` ($n = 3$ each). Best per config in bold.

Config	0.05	0.10	0.20	0.30	0.50
CPU12	**161.47**	161.96	162.59	162.24	162.76
GPU	**77.60**	77.81	77.82	77.83	78.03
GPU+11CPU	53.45	**52.80**	52.90	53.21	54.61

Tabla 2: Population standard deviations per configuration and `diffT`, in seconds.

Config	0.05	0.10	0.20	0.30	0.50
CPU12	0.47	0.71	0.32	0.94	2.44
GPU	0.03	0.08	0.08	0.09	0.21
GPU+11CPU	1.26	0.50	0.25	0.75	0.39

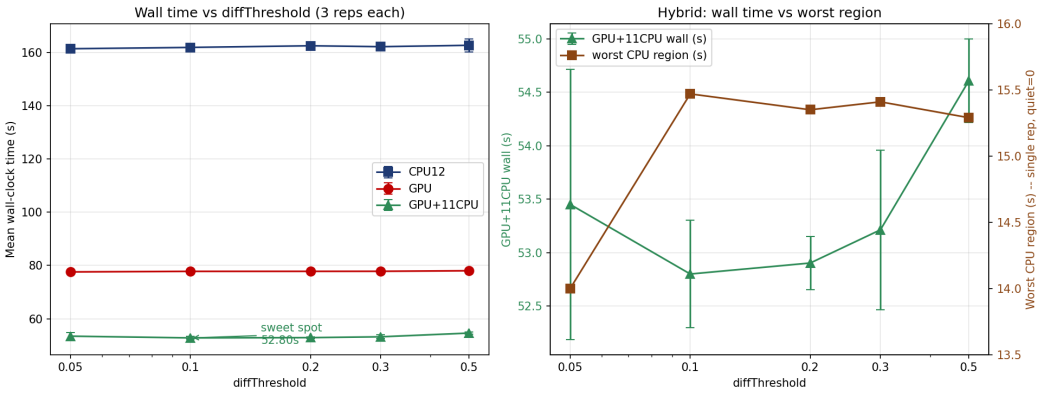

Figura 1: Left: mean wall time per config across `diffT` (log x-axis), error bars showing standard deviation. Right: zoom on the `GPU+11CPU` hybrid with the worst-case CPU region time overlaid on a secondary axis.

2.2. Work Generated vs. `diffT`

Tabla 3: Total leaf regions generated per run, averaged across 3 reps. The total is workload-independent and bounded above by the pixel-size floor.

Config	0.05	0.10	0.20	0.30	0.50
`CPU12` (CPU leaves)	5,323	5,323	5,320	5,290	5,152
`GPU` (GPU leaves)	5,323	5,323	5,320	5,290	5,152
`GPU+11CPU` CPU/GPU split	1381/3942	1334/3989	1334/3986	1408/3882	1339/3813
`GPU+11CPU` total	5,323	5,323	5,320	5,290	5,152

2.3. Worst-Case Region vs. `diffT`

The single `quiet=0` rep per `diffT` of `GPU+11CPU` records the maximum `[CPU region]` compute time and the pixel dimensions of that region:

Tabla 4: Worst-case CPU region per `diffT`, single `quiet=0` rep, `GPU+11CPU` configuration. Single-rep wall times are noisier than Table 1.

diffT	Wall (s)	Worst CPU region	Worst kernel (ms)	Worst kernel size
0.05	51.15	14.00 s @ 960 × 540	137.9	480 × 270
0.10	53.08	15.47 s @ 960 × 540	139.8	480 × 270
0.20	54.00	15.35 s @ 960 × 540	140.7	480 × 270
0.30	54.75	15.41 s @ 960 × 540	140.7	480 × 270
0.50	54.34	15.29 s @ 960 × 540	141.1	480 × 270

3. Analysis

3.1. The Hybrid Sweet Spot Is `diffT` = 0.1

The clearest finding is that the best hybrid configuration is not at the legacy default (0.5) nor at the most aggressive value (0.05) but at `diffT` = 0.1, with a mean of 52.80 s. Relative to the 0.5 baseline:

- 0.5 → 0.3: −1.40 s (−2.6%).
- 0.5 → 0.2: −1.71 s (−3.1%).
- 0.5 → 0.1: −1.81 s (−3.3%).
- 0.5 → 0.05: −1.16 s (−2.1%), but with σ = 1.26 s vs. 0.39 s at 0.5. The mean improved but the worst rep ran *worse* than the 0.5 baseline.

The point of inflection between 0.1 and 0.05 is the only non-monotonic piece of the curve. Past 0.1 the threshold no longer produces extra useful splits (the pixel-size floor takes over: total leaves saturate at exactly 5,323 by `diffT` = 0.1) but it does increase queue-scheduling jitter, which shows up in the variance.

3.2. Pure GPU Benefits Monotonically (But Tinily)

The `GPU` configuration improves from 78.03 s at 0.5 to 77.60 s at 0.05 — a 0.43 s (−0.5%) improvement that is small in fraction but real in σ units: the standard deviation at every `diffT` for `GPU` is below 0.1 s, so the 0.43 s gap is $> 5\sigma$ of either endpoint. The mechanism is the same as on the hybrid side — smaller, more uniform regions mean less time spent on the worst-case kernel launch — but with only one GPU thread there is no load-balance benefit to capture, only the modest reduction in tail kernel duration (from 141.1 ms to 137.9 ms; see Table 4).

3.3. CPU-Only Is Almost Invariant

`CPU12` runs in 161.5–162.8 s across the entire `diffT` range, with most of the variation absorbed by run-to-run noise. The mechanism: with 12 CPU threads sharing roughly 5,300 leaves of work totalling ∼2000 CPU-thread-seconds, the per-leaf granularity is already fine enough at any `diffT` for the dynamic queue to balance well. The bottleneck in CPU-only mode is total work-divided-by-threads, not load imbalance, so the small +3% leaves created by going from `diffT` = 0.5 to 0.05 adds 0.5% to the total compute time and saves a similar amount on the tail — a wash.

3.4. The Surprise: The Outlier Is Unbreakable

The worst single CPU region remains a 960 × 540 patch **at every `diffT` tested**, taking 14.0–15.5 s. Five points of `diffT` from 0.05 to 0.5 (a 10× range) move its compute time by less than 1.5 s. This is the central surprise of the experiment, and it is also the explanation for why all the preceding wall-time improvements are so small:

The corner-spread heuristic cannot see this region as a split candidate at any `diffT`. A 960 × 540 leaf is a quarter of an early-zoom frame; its four corners almost certainly all lie inside the Mandelbrot set, where `diverge()` returns the cap `MAXITER` = 10,000. With identical corner values, the spread `maxIter` − `minIter` = 0, and the decision rule `0 < diffThresh * maxIter` is true for every positive `diffThresh`.

The region is classified as uniform and dispatched to `compute()` in one block, where it sits on a single CPU thread for ∼15 s while the other threads complete the remaining ∼5,320 leaves and idle.

Lowering `diffT` does not help because the corners are *identical*, not just close. The diffT knob controls how much *near*-uniformity is acceptable; it cannot reject *exact* uniformity. The only way to subdivide this region with the existing heuristic is to lower `pixelSizeThresh` far enough that $960 \times 540 = 518{,}400$ pixels falls below the floor. That would force-split the region geometrically, which would attack the outlier but would also force-split the much larger interior of the set (a substantial fraction of every frame), generating overhead that very likely overwhelms the gain.

3.5. So Where Does the 3.3 % Hybrid Improvement Come From?

Not from shrinking the worst region, since it is essentially unchanged. The improvement comes from the other 5,320 leaves. Specifically, at `diffT` = 0.5 the renderer produces 5,152 leaves, but the bigger averaged-size of those leaves means the GPU thread spends slightly more time per kernel launch and the CPU threads spend slightly more time per region. At `diffT` = 0.1 the renderer produces 5,323 leaves — 171 more, +3.3% — each *slightly* smaller on average. Three effects compound:

1. **More uniform region sizes** mean the tail of the leaf distribution (the leaves the CPU threads chew on) is closer to the median, so any given CPU thread finishes its last leaf closer to the slowest thread's completion time. Per-thread wall times tighten.

2. **The GPU consumes leaves ∼40× faster than the CPU**, so the GPU thread is always returning to the queue looking for more work. More leaves at finer granularity means the GPU stays usefully busy for longer; its idle tail (the time it spends waiting for the last CPU thread) shrinks.

3. **The worst-case region's ∼15 s wall is masked by other workers**. When that region runs on, say, thread 7, the other 10 CPU threads continue chewing on the queue. The wall time becomes `max(15.3 s, time-to-drain-remaining-queue)`. More leaves means the queue takes slightly longer to drain, so the 15.3 s is more often *below* the queue-drain time and ceases to be the binding constraint for that rep.

The third effect explains why `diffT` = 0.05 has high variance: the queue-drain time becomes very close to the worst-region time, and small scheduling differences flip which side of that inequality dominates the rep. Sometimes the queue takes longer (⇒ stable, fast rep); sometimes it doesn't (⇒ the 15 s sets the wall and the rep is slow).

3.6. Why The Wall Improvement Plateaus

Five points of `diffT` change the wall by 1.81 s. The same order-of-magnitude as the noise in a single rep at the most aggressive threshold. The remaining 52–53 s of wall time is the irreducible constant on this platform with this binary: ∼15 s for the worst region plus ∼37 s of remaining-queue drain. To improve further the renderer needs:

- **To split the worst region**. The corner heuristic will never do this. Required: edge-midpoint sampling (the centre of the worst 960×540 region likely has a non-trivial iteration value; sampling it would detect interior non-uniformity) or a geometric/pixel-floor-only force-split for any region whose corners are all `MAXITER`.
- **To use cardioid and bulb interior certificates.** For truly-interior regions, this skips iteration entirely. The payoff in worst-case savings is bounded by however much of the outlier region is genuinely inside vs. just appearing to be from the corners.
- **Asynchronous CUDA streams**. Independent of the heuristic, recovers up to $\sim$13 ms of per-region GPU idle.

4. Recommendations

The cheapest configuration change available today is to lower the default `diffThreshold` from 0.5 to 0.1:

- Best-hybrid wall: 54.61 $\rightarrow$ 52.80 s (-3.3%).
- Pure-GPU wall: 78.03 $\rightarrow$ 77.81 s (-0.3%, but it is still GPU's best).
- CPU-only wall: unchanged within noise.
- Run-to-run variance for the hybrid actually *tightens* at 0.1 versus 0.5 ($\sigma = 0.50$ s vs. 0.39 s — nearly equal — but the $\sigma = 2.44$ s outlier observed on `CPU12` at 0.5 does not recur at 0.1).

Avoid `diffT` $= 0.05, which trades the small mean improvement for 2.5 \times$ higher run-to-run variance — predictable performance is more valuable than a marginal mean.

The diffT knob is now functionally exhausted on this workload. To shrink the wall time further the heuristic itself must be replaced or augmented. The three modifications listed at the end of `postfix_report.tex` (edge-midpoint sampling, cardioid certificates, async streams) remain the right candidates; this report now provides quantitative evidence for the urgency of the first one, since the 15 s outlier is demonstrably immune to any tightening of the existing rule.

5. Conclusion

The `diffThreshold` parameter has a real but modest effect on end-to-end wall time. For the best hybrid configuration on this machine, the sweet spot is `diffT` $= 0.1$ at 52.80 s, a 3.3 % improvement over the legacy default. The mechanism is better load balance among the non-outlier leaves, not elimination of any worst-case region.

The surprise of the experiment is that the worst-case CPU region — a single 960×540 patch consuming $\sim$15 s on one CPU thread — is invariant across the entire `diffT` sweep. Its four corners are all at `MAXITER`, making the corner-spread identically zero and the uniformity rule pass at every threshold. This region is the true binding constraint on the wall time, and no amount of `diffT` tuning can subdivide

it. The four-corner heuristic has reached its ceiling on this workload; meaningful future gains require either sampling more points inside each region (so that interior filaments cannot hide behind agreeing corners) or analytic interior tests (so that genuinely interior regions are recognised and constant-filled rather than iterated).